TickTock – Writeup

Prepared by: sebh24

Machine Author(s): Blitztide

Difficulty: Medium

Scenario

> Gladys is a new joiner in the company, she has recieved an email informing her that the IT department is due to do some work on her PC, she is guided to call the IT team where they will inform her on how to allow them remote access. The IT team however are actually a group of hackers that are attempting to attack Forela.

Description

> In this Sherlock, you find yourself investigating a potential spear-phishing attack against Forela. Gladys, a new employee, has received an email supposedly from the IT department, asking her to provide them with remote access to her PC for maintenance. Unbeknownst to Gladys, the 'IT team' is in reality a group of hackers posing as internal staff to infiltrate Forela's systems. Your mission is to assess the nature and extent of this attempted attack, identify any potential breaches in security that may have occurred, and establish defensive measures to prevent such occurrences in the future. This challenge will test your ability to analyze communication trails, spot phishing tactics, and effectively investigate a complex attack.

Artefacts Provided

Enter the artefacts provided along with their file hash here.

- ticktock.zip - *1cb5d3267271978ed0bd1eb0d667932e59a45aa8daa8a330e4fd62d6e80c6f4a*
  - Containing a KAPE output

Initial Analysis

After decompressing the zip file with the password `hacktheblue` we have a look at the artifacts provided to us.

```
sebh24@rabat:~/Git/def-content/TickTock/Artefacts/Image/C$ ls -alh
total 166M
drwxrwxr-x 7 sebh24 sebh24 4.0K May 10  2023  .
drwxrwxr-x 3 sebh24 sebh24 4.0K May 10  2023  ..
-rw-rw-r-- 1 sebh24 sebh24 8.0K May  4  2023 '$Boot'
drwxrwxr-x 3 sebh24 sebh24 4.0K May 10  2023 '$Extend'
-rw-rw-r-- 1 sebh24 sebh24  35M May  4  2023 '$LogFile'
-rw-rw-r-- 1 sebh24 sebh24 129M Oct 25  2022 '$MFT'
drwxrwxr-x 3 sebh24 sebh24 4.0K May 10  2023 '$Recycle.Bin'
-rw-rw-r-- 1 sebh24 sebh24 1.5M Oct 25  2022 '$Secure_$SDS'
drwxrwxr-x 3 sebh24 sebh24 4.0K May 10  2023  ProgramData
drwxrwxr-x 5 sebh24 sebh24 4.0K May 10  2023  Users
drwxrwxr-x 8 sebh24 sebh24 4.0K May 10  2023  Windows
```

The artefacts are consistent with a KAPE collection utilising Zimmermans tools and follow the same structure. KAPE (Kroll Artifact Parser and Extractor) is a forensic tool used for the rapid collection and processing of digital evidence from computers. It targets specific locations on a device for known artefacts relevant to digital investigations. The output we're seeing seems to be from a collection run by KAPE, focusing on certain Windows system files and directories that are commonly of interest in forensic investigations. Here's an explanation of each item listed in your output:

1. **`$Boot`** : This file is part of the NTFS file system used by Windows. It contains the boot sector information, which is necessary for the OS to start.
2. **`$Extend`** : This is a directory containing metadata files used by NTFS. It includes files like `$UsnJrnl` , which is the USN Journal (Update Sequence Number Journal), tracking changes to the volume.
3. **`$LogFile`** : This is the NTFS log file. It's used for recovery when the file system is not unmounted properly. This file logs transactions that occur on the NTFS volume, helping in the event of a system crash to restore consistency to the file system.
4. **`$MFT`** : The Master File Table is a critical file in NTFS, as it keeps records of all files and directories stored on the NTFS volume. Each file and directory has at least one entry in the MFT.

5. `$Recycle.Bin` : This is a folder where deleted files are temporarily stored in Windows. Files in the recycle bin can be restored to their original location by the user.
6. `$Secure_$SDS` : This file is part of the NTFS file system's security features, containing the security descriptor stream, which manages access controls and auditing settings for files and directories.
7. `ProgramData` : This directory stores application and system data that is not specific to a user account on the system.
8. `Users` : This directory contains the user profiles on the machine. Each user profile has personal files, settings, and configurations.
9. `Windows` : This is the main directory for the Windows operating system, containing system files, installed applications, and other system-related files.

As we are dealing with a KAPE output we understand that we'll potentially have to delve into the MFT and/or Windows Event Logs. A script can be utilised to use Zimmermans Tools, evtxecmd & mftecmd, to convert these into CSV for easier parsing into our tool sets.

```
# User Input for MFT file
$mftFile = Read-Host -Prompt 'Input the MFT file path'
# User Input for output directory
$outputDirectory = Read-Host -Prompt 'Input the output directory path'
# User Input for Event Logs directory
$eventLogsDirectory = Read-Host -Prompt 'Input the Event Logs directory
path'

# Ensure the output directory exists
if(!(Test-Path -Path $outputDirectory )) {
    Write-Host "The output directory does not exist. Creating it..."
    New-Item -ItemType Directory -Force -Path $outputDirectory
}

# Run MFTecmd
$mftOutputFile = Join-Path -Path $outputDirectory -ChildPath
"MFTOutput.csv"
```

```
& C:\Tools\Zimmerman-Tools\MFTECmd.exe --csv "$mftOutputFile" -f
"$mftFile"
Write-Host "MFTecmd completed. The output is saved at $mftOutputFile"

# Run evtxecmd
$evtxOutputFile = Join-Path -Path $outputDirectory -ChildPath
"EvtxOutput.csv"
& C:\Tools\Zimmerman-Tools\EvtxECmd\EvtxECmd.exe -d
"$eventLogsDirectory" --csv "$evtxOutputFile"
Write-Host "evtxecmd completed. The output is saved at $evtxOutputFile"
```

Following this I input the data into a Splunk instance I already have spun up on my physical host.

I have ingested all the data into an index named "ticktock".

Moving onto the next stage of analysis I want to get as close as possible to a process listing to understand what may have been used for initial access. We can do this by printing a list of processes found with sysmon as detailed below:

Interestingly Teamviewer has been executed numerous times, which is a legitimate and widely-used remote access tool, can become an attractive tool for threat actors, especially in the context of phishing incidents, for several reasons:

1. **Legitimacy and Trust**: TeamViewer is a legitimate software often used for remote support and access. This built-in trust can be exploited by attackers, as recipients might be less suspicious of a familiar tool.
2. **Ease of Remote Control**: Once installed, TeamViewer allows complete remote control of a system. An attacker can use this feature to execute commands, install additional malware, steal data, or move laterally within a network.

3. **Bypassing Security Measures**: Since TeamViewer is a legitimate application, it's less likely to be flagged or blocked by antivirus and firewall systems. This can help attackers maintain persistence and avoid detection.
4. **Simple Deployment**: Phishing emails can trick users into installing TeamViewer under the guise of legitimate requests, like tech support. The ease of installation and use means even less tech-savvy users can be tricked into granting access to their computers.
5. **Anonymity and Obfuscation**: Connections through TeamViewer can be difficult to trace back to the attacker. This anonymity is beneficial for threat actors wanting to hide their tracks and location.
6. **Phishing Integration**: In phishing scenarios, attackers might send emails impersonating IT support or a trusted authority, asking the recipient to install TeamViewer for a supposed legitimate purpose, like troubleshooting or software updates.
7. **Multi-Platform Support**: TeamViewer works across various platforms (Windows, macOS, Linux), broadening the range of potential targets for an attacker.
8. **Data Exfiltration**: Once control is gained, TeamViewer can be used to transfer files from the victim's computer, facilitating data theft.
9. **Screen and Input Capture**: Attackers can use TeamViewer to monitor user activity, capture keystrokes, or take screenshots, gathering sensitive information or credentials.

Teamviewer log data is usually stored under the relevant user's AppData directory. We locate the Teamviewer log file within `\ticktock\Collection\C\Users\gladys\AppData\Local\TeamViewer\Logs`. Upon opening the log file we locate evidence of the download of our known malicious executable "merlin.exe".

```
2023/05/04 11:21:30.996  4428       6012 G3   Write file
C:\Users\gladys\Desktop\merlin.exe
2023/05/04 11:21:34.398  4428       6012 G3   Download from "merlin.exe"
to "C:\Users\gladys\Desktop\merlin.exe" (10.95 MB)
```

Our next step is to confirm which connection this occured within:

```
2023/05/04 11:35:27.433  5716       5840 D3
SessionManagerDesktop::IncomingConnection: Connection incoming,
sessionID = -2102926010
2023/05/04 11:35:27.433  5716       5840 D3
CParticipantManagerBase::SetMyParticipantIdentifier(): pid=
[1764218403,-2102926010]
2023/05/04 11:35:27.434  5716       5840 D3!!
InterProcessBase::ProcessControlCommand Command 39 not handled
2023/05/04 11:35:27.434  5716       5840 D3   IpcRouterClock: received
router time: 20230504T103558.360315
2023/05/04 11:35:27.435  5716       4292 D3   CLogin::run(), session id:
-2102926010
2023/05/04 11:35:27.436  5716       4292 D3
CLogin::NegotiateVersionServer()
2023/05/04 11:35:27.559  5716       4292 D3   BuddyLoginToken 1,
SupporterDataRequest 1
2023/05/04 11:35:27.918  5716       4292 D3
CLoginServer::CheckIfConnectionIsAllowed()
2023/05/04 11:35:28.169  5716       5840 D3
LoginServer::HandleBlockConditionsResponse: using condition set: {
(true) -> 11111111111111111111 }
2023/05/04 11:35:28.169  5716       4292 D3
CLoginServer::AuthenticateServer()
2023/05/04 11:35:28.170  5716       4292 D3
InterProcessBase::SecureNetworkCallbackHandle created (RegistrationID:
5d3a67ff-f76f-472d-9b82-68674bdff9ea)
2023/05/04 11:35:31.669  5716       5840 D3
AuthenticationSmartAccess_Passive::NextStep: user confirmation OK
2023/05/04 11:35:31.669  5716       5840 D3
AuthenticationInstantSupport_Passive::RunAuthenticationMethod:
authentication was successful
2023/05/04 11:35:31.670  5716       4292 D3
InterProcessBase::SecureNetworkCallbackHandle destroyed (RegistrationID:
5d3a67ff-f76f-472d-9b82-68674bdff9ea)
2023/05/04 11:35:31.772  5716       4292 D3   CLoginServer::runServer:
ConnectionMode == 1
```

```
2023/05/04 11:35:31.772  5716       4292 D3
SessionManagerDesktop::ChangeToServermode: creating session with
tvsessionprotocol::TVSessionID = -2102926010
2023/05/04 11:35:31.772  5716       4292 D3
SessionManagerDesktop::InitiateDesktopSession: creating session with
tvsessionprotocol::TVSessionID = -2102926010
2023/05/04 11:35:31.773  5716       5840 D3
SessionManagerDesktop::ApplyInputBlockerPermissions: apply permissions:
0
2023/05/04 11:35:31.774  5716       4292 D3
WorkstationLockerWin::ShouldAutoLockWorkstation: Autolock: no, Local
user logged-in: 1, window session locked: 0, secure screen saver
running: no disabled by policy: 0
```

We can now confirm that the initial access by the TA occured at 2023-05-04 11:35:27 . This is confirmed by the Incoming Connection line, which also confirms the unique session ID -2102926010.

We are also able to pull out additional IOCs within the log file such as:

- `2023/05/04 11:35:31.958  5716       2436 D3   CParticipantManagerBase participant fritjof olfasson (ID [1761879737,-207968498]) was added with the role 6`

This confirms the assumed name for the potential threat actor who dropped merlin.exe onto the endpoint. If we move over to our MFT file, which is also ingested into Splunk we can confirm the time the file was dropped onto the host.

```
index=ticktock source="20231213143222_MFTECmd_$MFT_Output.csv" merlin
ParentPath=".\\Users\\gladys\\Desktop" | table  FileName Created0x10
LastModified0x10
```

At this stage we are aware the attacker gained access utilising Teamviewer followed by the deployment of a C2 agent - Merlin. We can confirm within our Event Logs that Sysmon is installed on the endpoint by listing the Channels.

```
index=ticktock source="20231213143328_EvtxECmd_Output.csv" | stats count by Channel | Table Channel
```

A good place to look for any potential command line activity carried out by the attacker would be within the sysmon data. At this stage we are aware Merlin was utilised.

Next we want to confirm if any child processes were executed utilising Merlin.

```
index=ticktock source="20231213143328_EvtxECmd_Output.csv" Channel="Microsoft-Windows-Sysmon/Operational" merlin PayloadData4="ParentProcess: C:\\Users\\gladys\\Desktop\\merlin.exe"
```

We are able to locate a directory created utilsing cmd.exe C:\temp. Next we attempt to locate any potential execution from within this directory however are unable to. We then look at all events within a similar time frame using Splunks inbuilt feature:

Interestingly we locate multiple events indicating system time changes, which are unusual for the normal operation of a system and they exist within the period of the compromise.

We locate a total of 4749 system time change events split between 2 x log Channels:

- Security
- System

If we run the following search

```
index=ticktock  MapDescription="The system time was changed" | stats count by ExecutableInfo
```

we are able to seperate the system changes by Powershell.

Confirming the system time was infact changed 2371 times by Powershell. The question within the platform references a script. Whilst here we additionally confirm the SID of the user running - `[S-1-5-21-3720869868-2926106253-3446724670-1003]` .

At this phase of our analysis it is important to take the event log times with a pinch of salt, making timelining very difficult. Confirming that numerous time changes have occurred utilising Powershell, this corresponds to a file found in the MFT during MFT analysis. We attempt to locate any PowerShell scripts on the file system under Glady's user profile.

```
index=ticktock ParentPath!=*KAPE* "Users\\gladys" source="20231213143222_MFTECmd_$MFT_Output.csv" .ps1 | table FileName Created0x10 LastModified0x10
```

Based on both the creation time and the expected time of compromise it is highly likely that the Invoke-TimeWizard.ps1 was utilised by the TA to adjust the system time.

Questions

1. What was the name of the executable that was uploaded as a C2 Agent?
   `merlin.exe`

   Within our Splunk host we perform an initial search across the field 'PayloadData6'. The question indicates that there is a binary on the endpoint which is utilising Command and Control (C2) communications. This would indicate it likely regularly communicates with an external IP address. We perform a search in Splunk to locate any top talking IPs within the event logs.

   The IP 52.56.142.81 is found communicating within the sysmon logs out of the network 357 times. This is the top talker by far. We now want to further validate this.

If we take some of the raw data from one of the events with regards to the potentially suspicious IP as detailed below:


```
2913,2913,2023-05-04 12:37:33.5167203,3,Info,Microsoft-Windows-
Sysmon,Microsoft-Windows-Sysmon/Operational,2980,472,DESKTOP-
R30EAMH,44,S-1-5-18,Network connection,DESKTOP-
R30EAMH\gladys,,"ProcessID: 5768, ProcessGUID: 5080714d-89ce-6453-
c202-000000000700",RuleName: Usermode,SourceHostname: DESKTOP-
R30EAMH.forela.local,SourceIp: 10.10.0.79,DestinationHostname: ec2-
52-56-142-81.eu-west-2.compute.amazonaws.com,DestinationIp:
52.56.142.81,,False,C:\Users\sebh24\Desktop\ticktock\Collection\C\Wi
ndows\System32\winevt\logs\Microsoft-Windows-
Sysmon%4Operational.evtx,Classic,0,"{""EventData"":{""Data"":
[{""@Name"":""RuleName"",""#text"":""Usermode""},
{""@Name"":""UtcTime"",""#text"":""2023-05-03 11:38:16.932""},
{""@Name"":""ProcessGuid"",""#text"":""5080714d-89ce-6453-c202-
000000000700""},{""@Name"":""ProcessId"",""#text"":""5768""},
{""@Name"":""Image"",""#text"":""C:\\Users\\gladys\\Desktop\\merlin.
exe""},{""@Name"":""User"",""#text"":""DESKTOP-R30EAMH\\gladys""},
{""@Name"":""Protocol"",""#text"":""tcp""},
{""@Name"":""Initiated"",""#text"":""True""},
{""@Name"":""SourceIsIpv6"",""#text"":""False""},
{""@Name"":""SourceIp"",""#text"":""10.10.0.79""},
{""@Name"":""SourceHostname"",""#text"":""DESKTOP-
R30EAMH.forela.local""},
{""@Name"":""SourcePort"",""#text"":""50636""},
{""@Name"":""SourcePortName"",""#text"":""-""},
{""@Name"":""DestinationIsIpv6"",""#text"":""False""},
{""@Name"":""DestinationIp"",""#text"":""52.56.142.81""},
{""@Name"":""DestinationHostname"",""#text"":""ec2-52-56-142-81.eu-
west-2.compute.amazonaws.com""},
{""@Name"":""DestinationPort"",""#text"":""80""},
{""@Name"":""DestinationPortName"",""#text"":""http""}]}}"
```

We are able to locate the image name "merlin.exe", indicating the existence of a known command and control agent on the endpoint. Merlin.exe is a sophisticated Command-and-Control (C2) malware commonly used in cyber-espionage and targeted attacks. This executable serves as a critical component in a threat actor's arsenal, enabling remote control over compromised systems. At this stage its also important to note some information such as the process ID, destination IP & hostname:

- Destination IP: 52.56.142.81
- Destination hostname: ec2-52-56-142-81.eu-west-2.compute.amazonaws.com
- Process ID: 5768
- Merlin Location on FS: C:\Users\gladys\Desktop\merlin.exe
- User running: Gladys

Expanding on the hunt for Merlin.exe we perform a search within Splunk for any detection relevant to Merlin within the Microsoft-Windows-Windows Defender/Operational channel. Due to Merlin being an off the shelf C2 agent, with the default name assigned it is likely Defender alerted.

```
index=ticktock merlin.exe Channel="Microsoft-Windows-Windows Defender/Operational" | table _time EventId PayloadData1, PayloadData2, PayloadData3
```

We confirm there were infact alerts relevant to Merlin.exe by Defender, however an exclusion was placed. The name assigned was `VirTool:Win32/Myrddin.D`

2. What was the session id for in the initial access?

   `-2102926010`

   Answered in previous analysis within the Teamviewer logs.

3. The attacker attempted to set a bitlocker password on the `C:` drive what was the password?

   `reallylongpassword`

4. What name was used by the attacker?

   `fritjof olfasson`

5. What IP address did the C2 connect back to?

`52.56.142.81`

Answered within previous analysis.

6. What category did Windows Defender give to the C2 binary file?

   `VirTool:Win32/Myrddin.D`

   Answered within previous analysis.

7. What was the filename of the powershell script the attackers used to manipulate time?

   `Invoke-TimeWizard.ps1`

   Answered within initial analysis.

   However continuing the further confirmation of execution can be achieved by reading the PSReadline log within `\Users\gladys\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\`

   ```
   set-executionpolicy bypass
   cd ..
   cd ..
   cd .\Users\
   cd .\gladys\Desktop\
   dir
   .\Invoke-TimeWizard.ps1
   ```

8. What time did the initial access connection start? (UTC)

   `04/05/2023 11:35:27`

   Answered within initial analysis.

9. What is the SHA1 and SHA2 sum of the malicious binary?

   `ac688f1ba6d4b23899750b86521331d7f7ccfb69:42ec59f760d8b6a50bbc7187829f62c3b6b8e1b841164e7185f497eb7f3b4db9`

   We able to locate both the SHA1 & SHA2 sums of the malicious binary within `ticktock\Collection\C\ProgramData\Microsoft\Windows Defender\Support` due to the previous Defender alerts. When defender fires an alert the SHA1 & 2 sums of the malicious file are logged within MPLog-07102015-052145 log file.

```
IsNotFpCheckDisabledSig=true, IsSignedFileCheck=false,
IsNotExcludedCertificate=true (FriendlySigSeq=0x0)
SDN:Issuing SDN query for \\?\C:\Users\gladys\Desktop\merlin.exe
(\\?\C:\Users\gladys\Desktop\merlin.exe)
(sha1=ac688f1ba6d4b23899750b86521331d7f7ccfb69,
sha2=42ec59f760d8b6a50bbc7187829f62c3b6b8e1b841164e7185f497eb7f3b4db
9)
SDN:SDN query completed: 00000000
```

10. How many times did the powershell script change the time on the machine?

    `2371`

    Answered within initial analysis.

11. What is the SID of the victim user?

    `S-1-5-21-3720869868-2926106253-3446724670-1003`

    Answered within initial analysis.